

Université Cadi Ayyad — FST Marrakech
Master IAI — Intelligence Artificielle et Ingénierie Informatique

Rapport de Projet

Pipeline ML de Classification du Cancer du Côlon

Déploiement Contenarisé avec Docker & FastAPI

Module	Gestion de Projet et Méthodologie de Recherche en IA
Encadrante	Mme O. Banouar
Réalisé par	Ahmed Mohati
Filière	M1 IAI
Année universitaire	2025 — 2026

1. Introduction et Contexte

Ce rapport présente la conception et la mise en œuvre d'un pipeline complet de Machine Learning appliqué à la classification du cancer du côlon à partir de données d'expression génomique. Le projet s'inscrit dans une démarche de recherche reproductible et de déploiement industriel, en intégrant les bonnes pratiques de gestion de projet logiciel.

Le cancer du côlon constitue l'une des pathologies oncologiques les plus fréquentes dans le monde. L'utilisation de données génomiques à haute dimension (microarray) pour sa détection automatique représente un défi majeur en apprentissage automatique, notamment en raison du rapport $n \ll p$ (peu d'échantillons, très grande dimensionnalité).

2. Description du Dataset

2.1 Caractéristiques générales

Caractéristique	Valeur	Remarque
Nombre d'échantillons	62	Très petit dataset
Nombre de features	2 001	Expressions géniques
Classe Abnormal	40 (64,5 %)	Classe majoritaire
Classe Normal	22 (35,5 %)	Classe minoritaire
Valeurs manquantes	0	Dataset propre

2.2 Problèmes identifiés

- Déséquilibre de classes (ratio 1,8:1) — risque de biais vers la classe majoritaire
- Haute dimensionnalité (2001 features, 62 échantillons) — curse of dimensionality
- Nécessité de normalisation — plages de valeurs très larges (1 000 – 15 000)

3. Pipeline de Machine Learning

3.1 Stratégie retenue

Plusieurs stratégies ont été comparées pour résoudre l'overfitting (train accuracy = 1,0 / test accuracy = 1,0 observé sans preprocessing). Le tableau suivant synthétise les résultats obtenus :

Stratégie	Train Acc.	Test Acc.	Retenu ?
PCA (10 composantes)	0,939	0,692	Non
PCA (20 composantes)	0,959	0,769	Non
SelectKBest (k=20)	0,857	0,769	Non
SelectKBest + SMOTE	0,938	0,769	Non
SelectKBest + class_weight='balanced' ✓	0,918	0,769	✓ Oui

La stratégie **SelectKBest(k=20) + Logistic Regression (class_weight='balanced')** a été retenue car elle offre de meilleures garanties d'interprétabilité : les 20 gènes sélectionnés sont nommés et biologiquement interprétables, contrairement aux composantes PCA.

3.2 Étapes du pipeline

- StandardScaler : normalisation z-score de l'ensemble des 2 001 features
- SelectKBest (ANOVA F-test, k=20) : sélection des 20 gènes les plus discriminants
- Train/Test split : 80/20, stratifié par classe
- LogisticRegression (C=0,01, solver=lbfgs, class_weight='balanced') : classification binaire

3.3 Résultats finaux

Métrique	Valeur
Train Accuracy	91,84 %
Test Accuracy	76,92 %
CV-5 Accuracy	88,46 % ($\pm$ 8,55 %)
AUC-ROC	0,90

3.4 Top 10 gènes sélectionnés (ANOVA F-test)

Rang	Gène	F-score	Pertinence
1	M63391	39,81	Très haute
2	M76378__	33,15	Très haute
3	R87126	32,02	Très haute
4	J02854	31,76	Très haute
5	M76378	30,95	Haute
6	M76378_	29,64	Haute
7	Z50753	25,34	Haute
8	T92451	24,81	Haute
9	U25138	20,54	Moyenne
10	H08393	19,44	Moyenne

4. Architecture de Déploiement

4.1 Vue d'ensemble

Le projet adopte une architecture microservices à deux conteneurs Docker orchestrés par Docker Compose, avec un volume partagé pour la transmission des artefacts ML entre les services.

Service	Rôle	Technologies	Statut
ml_service	Entraînement du modèle, sauvegarde des artefacts	Python, scikit-learn, pandas	Exécution unique
api_service	Exposition REST des prédictions et métriques	FastAPI, uvicorn, scikit-learn	Continu (:8000)

4.2 Flux Docker Compose

- `ml_service` démarre en premier et entraîne le modèle
- Les artefacts (`scaler.pkl`, `selector.pkl`, `model.pkl`, `metadata.json`) sont sauvegardés dans le volume partagé `colon_models_volume`
- `ml_service` se termine avec succès
- `api_service` démarre uniquement après (condition : `service_completed_successfully`)
- `api_service` charge les artefacts et expose l'API sur `http://localhost:8000`

4.3 Structure du projet

`colon_cancer_project/`

```
├─ docker-compose.yml    ← orchestration + volume partagé
├─ Makefile              ← commandes make up / push / test
├─ test_api.py           ← tests automatisés des endpoints
├─ data/colon_cancer_dataset.csv
├─ ml_service/           ← train.py + Dockerfile + requirements
└─ api_service/         ← main.py + Dockerfile + requirements
```

5. API REST — Endpoints

Méthode	Endpoint	Description	Réponse clé
GET	<code>/health</code>	Statut du service et du modèle	<code>status: ready</code>
POST	<code>/predict</code>	Prédiction sur un vecteur génomique	<code>prediction</code> , <code>probabilités</code> , <code>confidence</code>
GET	<code>/top-genes</code>	Top N gènes (F-score ANOVA)	<code>rank</code> , <code>gene</code> , <code>f_score</code>
GET	<code>/model-info</code>	Métriques et paramètres du modèle	<code>accuracy</code> , <code>AUC</code> , <code>confusion matrix</code>
GET	<code>/docs</code>	Documentation Swagger UI interactive	Interface web

6. Publication sur Docker Hub

Les deux images Docker ont été publiées sur Docker Hub sous le compte ahmedmohati, permettant un déploiement immédiat sur toute machine disposant de Docker sans nécessiter de code source ni d'environnement Python local.

Service	Image Docker Hub	URL
ml_service	ahmedmohati/colon-ml-trainer:latest	Docker Hub — colon-ml-trainer
api_service	ahmedmohati/colon-api:latest	Docker Hub — colon-api

6.1 Commandes de déploiement depuis Docker Hub

```
# Sur n'importe quelle machine
docker pull ahmedmohati/colon-ml-trainer:latest
docker pull ahmedmohati/colon-api:latest
docker compose up      # lancement complet
```

6.2 Différence local vs Docker Hub

Critère	Exécution locale	Docker Hub
Installation	Python + packages requis	Docker uniquement
Portabilité	Dépend de l'OS et des versions	Identique sur toute machine
Partage	Code source nécessaire	Image seule suffisante
Reproductibilité	Variable selon l'environnement	Garantie (image figée)
Déploiement cloud	Configuration manuelle	Pull & run immédiat

7. Conclusion

Ce projet a permis de mettre en pratique l'ensemble du cycle de vie d'un projet ML industriel : exploration et diagnostic des données, comparaison de stratégies de preprocessing, entraînement et évaluation rigoureuse, exposition via API REST, et déploiement conteneurisé reproductible.

La stratégie SelectKBest + Régression Logistique avec `class_weight='balanced'` a permis de réduire efficacement l'overfitting tout en conservant l'interprétabilité biologique des features sélectionnées. La publication sur Docker Hub finalise la démarche en garantissant la reproductibilité et la portabilité du pipeline sur tout environnement.

Les compétences mobilisées couvrent la gestion de projet logiciel (pipeline structuré, versionnement), la méthodologie de recherche en IA (comparaison expérimentale, validation croisée, métriques adaptées au déséquilibre), et les pratiques DevML (Docker, FastAPI, volume partagé, health checks).